Quantum Kernels in Practice

QML

Contents

1 Overview

last lecture we built up the theory of quantum kernels — quantum feature maps, the kernel trick in hilbert space, and the connection between variational circuits and kernel methods. now its time to actually use all that. this lecture is the practical follow-up: we go from theory to implementation.

we r going to walk through the full pipeline of implementing quantum kernels in qiskit, using them with scikit-learn SVMs, and benchmarking them against classical kernels on real (well, toy) datasets. this is the lecture where u actually learn how to build smtg that runs.

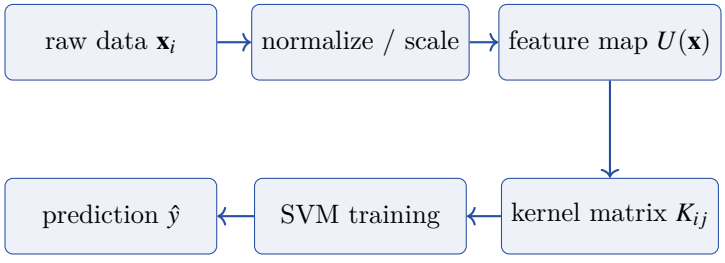
the main topics:

- how to implement quantum kernels step by step in qiskit
- the `FidelityQuantumKernel` class and wt it does internally
- choosing feature maps — which ones work, which ones dont, and why
- computing the kernel matrix in practice — shots, noise, statistical errors
- plugging quantum kernels into sklearn's SVM (QSVC, QSVR)
- kernel alignment — the metric that tells u if ur kernel is any good
- practical considerations: qubits, depth, noise
- benchmarking on iris, moons, and circles datasets

tbh this is where quantum kernels either impress u or disappoint u. spoiler: on small toy datasets, they work fine but dont beat classical methods. the real question is whether they can do smtg classical kernels cant on structured problems.

2 The Full Quantum Kernel Pipeline

before we dive into code details, lets see the big picture. the full workflow from raw data to prediction looks like this:



each step matters. if u mess up the normalization, ur feature map wont work well. if u choose a bad feature map, ur kernel matrix will be garbage. if u dont use enough shots, ur kernel entries will be noisy. lets go through each piece.

2.1 Step 1: Data Preprocessing

quantum feature maps typically encode data into rotation angles. this means ur data needs to be in a range thats compatible with rotation gates — usually $[0, 2\pi]$ or $[-\pi, \pi]$.

Key Point. always normalize ur data before feeding it to a quantum kernel. the standard approach is min-max scaling to $[0, \pi]$ or standardization followed by a sigmoid-like compression. if u skip this, ur rotation gates will either saturate (everything maps to roughly the same state) or underuse the available range.

the preprocessing pipeline:

1. start with raw features $\mathbf{x} \in \mathbb{R}^d$
2. apply standard scaling: $\mathbf{x} \to \frac{\mathbf{x}-\mu}{\sigma}$
3. rescale to $[0, \pi]$: $\mathbf{x} \to \pi \cdot \frac{\mathbf{x}-\mathbf{x}_{\min}}{\mathbf{x}_{\max}-\mathbf{x}_{\min}}$
4. if $d > n_{\text{qubits}}$, apply PCA to reduce dimensionality first

2.2 Step 2: Define the Feature Map Circuit

the feature map $U(\mathbf{x})$ is a parameterized quantum circuit that encodes classical data into quantum states. in qiskit, u create this using the circuit library.

the standard approach: create a `ZZFeatureMap` or `ZFeatureMap` from `qiskit.circuit.library`. u specify the number of qubits, the number of repetitions (called "reps"), and the entanglement pattern.

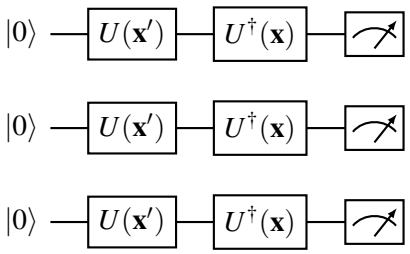
Definition 2.1 (Quantum Feature Map Circuit). a quantum feature map is a unitary $U(\mathbf{x})$ applied to $|0\rangle^{\otimes n}$ to produce the quantum state

$$|\phi(\mathbf{x})\rangle = U(\mathbf{x})\,|0\rangle^{\otimes n}$$

the corresponding quantum kernel is

$$\kappa(\mathbf{x}, \mathbf{x}') = |\,\langle\phi(\mathbf{x})|\phi(\mathbf{x}')\rangle\,|^2 = |\,\langle 0|^{\otimes n} U^\dagger(\mathbf{x}) U(\mathbf{x}')\,|0\rangle^{\otimes n}\,|^2$$

here is the circuit that computes this kernel entry:



the kernel value is the probability of measuring $|0\rangle^{\otimes n}$ at the output. if the two data points r similar, the states r similar, the inverse maps back close to $|0\rangle^{\otimes n}$, and u get a high probability. if theyre different, u get a low probability.

3 The FidelityQuantumKernel Class

qiskit provides the `FidelityQuantumKernel` class (from `qiskit_machine_learning.kernels`) that handles all the circuit construction and execution for u. lets understand wt it does internally.

3.1 What It Does

when u create a `FidelityQuantumKernel`, u pass it:

- a feature map circuit (e.g., `ZZFeatureMap`)
- a fidelity instance (computes the overlap between quantum states)

when u call `kernel.evaluate(X_train, X_test)`, it:

1. takes every pair $(\mathbf{x}_i, \mathbf{x}_j)$ from the input arrays
2. constructs the circuit $U^\dagger(\mathbf{x}_i)U(\mathbf{x}_j)$ for each pair
3. executes all circuits (possibly in batches)
4. extracts the probability of measuring all-zeros from each circuit
5. assembles these into the kernel matrix K_{ij}

Key Point. the `FidelityQuantumKernel` is essentially an automation layer. it loops over all pairs, builds circuits, runs them, and collects results. theres no magic — its doing exactly wt we described in the theory lecture, but handling the bookkeeping.

3.2 Internal Fidelity Computation

the fidelity between two quantum states can be computed in different ways:

Definition 3.1 (State Fidelity). for pure states, the fidelity is

$$F(\rho, \sigma) = |\langle\phi|\psi\rangle|^2$$

for the kernel, with $|\phi\rangle = U(\mathbf{x})|0\rangle^n$ and $|\psi\rangle = U(\mathbf{x}')|0\rangle^n$:

$$\kappa(\mathbf{x}, \mathbf{x}') = F(|\phi(\mathbf{x})\rangle, |\psi(\mathbf{x}')\rangle) = |\langle 0|^n U^\dagger(\mathbf{x})U(\mathbf{x}')|0\rangle^n|^2$$

qiskit offers two fidelity implementations:

- ComputeUncompute: constructs $U(\mathbf{x}')$ followed by $U^\dagger(\mathbf{x})$, measures in computational basis. this is the standard approach.
- StateFidelity via Sampler: uses the sampler primitive to estimate the overlap. more flexible but essentially does the same thing.

the compute-uncompute approach doubles the circuit depth bcs u need both $U(\mathbf{x}')$ and $U^\dagger(\mathbf{x})$. for a feature map with depth D, the kernel circuit has depth $2D$. this is important for noise analysis later.

3.3 Symmetry and Caching

the kernel matrix is symmetric: $K_{ij} = K_{ji}$. a good implementation exploits this and only computes $\frac{M(M+1)}{2}$ entries instead of M^2 (where M is the number of data points). the diagonal entries r always 1 (a state has perfect fidelity with itself), so really u need $\frac{M(M-1)}{2}$ circuit evaluations.

Intuition. think of the kernel matrix computation as filling in the upper triangle of a symmetric matrix. each entry requires running a quantum circuit with some number of shots. for $M = 100$ data points, thats roughly 5000 circuit evaluations. if each uses 1000 shots, thats 5 million circuit executions total. this is why quantum kernels r expensive.

4 Choosing Feature Maps

the feature map is the most important design choice. it determines wt the quantum kernel actually computes and whether it has any advantage over classical kernels. lets go through the main options.

4.1 ZFeatureMap

the simplest feature map. applies single-qubit rotations based on data features, with no entanglement.

for n qubits and data $\mathbf{x} = (x_1, \ldots, x_n)$:

$$U_Z(\mathbf{x}) = \prod_{r=1}^{\text{reps}} \left[\bigotimes_{i=1}^{n} H_i \cdot \bigotimes_{i=1}^{n} R_Z(x_i)_i \right]$$

where H is hadamard and $R_Z(\theta) = e^{-i\theta Z/2}$.

Intuition. the ZFeatureMap with 1 rep is basically a product state feature map. each qubit encodes one feature independently. the resulting kernel is a product of individual qubit overlaps:

$$\kappa_Z(\mathbf{x}, \mathbf{x}') = \prod_{i=1}^{n} \cos^2\left(\frac{x_i - x_i'}{2}\right)$$

this is equivalent to a classical cosine kernel. no quantum advantage here.

4.2 ZZFeatureMap

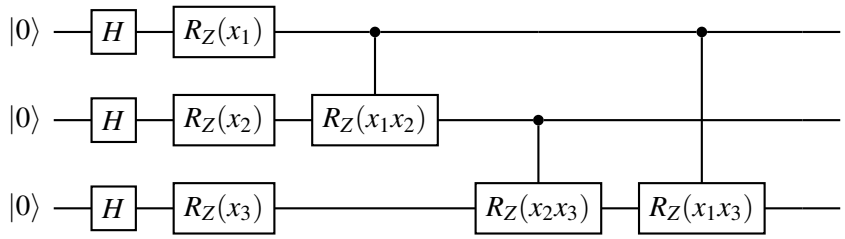
the standard choice for quantum kernels. adds entangling gates that create correlations between features.

$$U_{ZZ}(\mathbf{x}) = \prod_{r=1}^{\text{reps}} \left[\bigotimes_{i=1}^{n} H_i \cdot \prod_{i<j} e^{-ix_i x_j Z_i Z_j} \cdot \bigotimes_{i=1}^{n} e^{-ix_i Z_i} \right]$$

the ZZ interaction terms $e^{-ix_i x_j Z_i Z_j}$ create entanglement that depends on products of features. this is where the quantum part actually matters.

Key Point. the ZZFeatureMap produces kernels that cannot be efficiently computed classically (under reasonable complexity assumptions) when the circuit depth is sufficient. the paper by havlíček et al. (2019) showed this rigorously for a specific choice of feature map.

the circuit for a 3-qubit ZZFeatureMap (1 rep):



4.3 PauliFeatureMap

the most general option. u can specify which pauli operators to use for encoding:

$$U_P(\mathbf{x}) = \prod_{r=1}^{\text{reps}} \left[\bigotimes_{i=1}^{n} H_i \cdot \prod_{S \subseteq \{1,\ldots,n\}} e^{-i\phi_S(\mathbf{x}) \otimes_{j \in S} P_j} \right]$$

where $P_j \in \{I, X, Y, Z\}$ and $\phi_S(\mathbf{x}) = \prod_{j \in S} x_j$ is the data encoding function.

common paulis choices:

- `['Z']`: equivalent to ZFeatureMap
- `['ZZ']`: equivalent to ZZFeatureMap
- `['Z', 'ZZ']`: both single and two-qubit terms
- `['ZZ', 'ZZZ']`: two and three-qubit correlations
- `['Y', 'ZZ']`: mixing different pauli types

4.4 Feature Map Comparison

Feature Map	Entanglement	Depth	Expressivity	Classical?
ZFeatureMap	none	$O(\text{reps})$	low	yes
ZZFeatureMap	pairwise	$O(n^2 \cdot \text{reps})$	medium-high	no (conj.)
PauliFeatureMap (ZZZ)	three-body	$O(n^3 \cdot \text{reps})$	high	no (conj.)
Custom (hardware eff.)	varies	varies	varies	depends

Key Point. the rule of thumb: u need entangling gates for the kernel to be potentially hard to simulate classically. but more entanglement isnt always better — too much entanglement can make the kernel concentrate (all off-diagonal entries go to zero), which is useless for classification.

4.5 Entanglement Patterns

for ZZFeatureMap and PauliFeatureMap, u also choose the entanglement pattern:

- full: every pair of qubits is entangled. $O(n^2)$ gates per layer.

- linear: nearest-neighbor entanglement. $O(n)$ gates per layer.
- circular: like linear but with an extra gate connecting first and last qubit.
- sca (shifted circular alternating): alternates between even-odd and odd-even pairs.

for small numbers of qubits ($n \leq 6$), full entanglement is fine. for larger circuits, linear or circular is more practical bcs it keeps the circuit shallow and maps better to hardware connectivity.

5 Computing the Kernel Matrix

once u have ur feature map, u need to actually compute the kernel matrix. this is where the rubber meets the road — and where things get expensive.

5.1 Circuit Execution

for M training points, u need to evaluate:

$$K_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j) = |\langle 0|^n U^\dagger(\mathbf{x}_i) U(\mathbf{x}_j) |0\rangle^n|^2$$

total number of unique circuits: $\frac{M(M-1)}{2}$ (exploiting symmetry and known diagonal).

each circuit is run with N_s shots to estimate the probability. the estimated kernel entry is:

$$\hat{K}_{ij} = \frac{\text{number of all-zeros outcomes}}{N_s}$$

5.2 Statistical Error Analysis

each kernel entry is estimated from a binomial distribution. if the true kernel value is κ_{ij}, then after N_s shots:

Theorem 5.1 (Kernel Estimation Error). the estimated kernel entry $\hat{K}_{ij}$ satisfies

$$\mathbb{E}[\hat{K}_{ij}] = \kappa_{ij}, \qquad \text{Var}[\hat{K}_{ij}] = \frac{\kappa_{ij}(1-\kappa_{ij})}{N_s}$$

the standard error is

$$\text{SE}(\hat{K}_{ij}) = \sqrt{\frac{\kappa_{ij}(1-\kappa_{ij})}{N_s}}$$

which is maximized when $\kappa_{ij} = 0.5$, giving $\text{SE}_{\max} = \frac{1}{2\sqrt{N_s}}$.

Example 5.1. with $N_s = 1000$ shots:

- maximum standard error: $\frac{1}{2\sqrt{1000}} \approx 0.016$
- for $\kappa_{ij} = 0.9$: $\text{SE} = \sqrt{0.09/1000} \approx 0.0095$
- for $\kappa_{ij} = 0.1$: $\text{SE} = \sqrt{0.09/1000} \approx 0.0095$

so with 1000 shots, ur kernel entries r accurate to about ± 0.02 (two standard errors for 95% confidence).

Intuition. more shots = more accurate kernel entries = better SVM performance. but more shots also means more circuit executions. theres a tradeoff. in practice, 1000–8192 shots per circuit is a reasonable range. below 100 shots, the kernel matrix gets too noisy to be useful.

5.3 Positive Semi-Definiteness

a valid kernel matrix must be positive semi-definite (PSD). the true quantum kernel is always PSD by construction (its a gram matrix of inner products). but the estimated kernel matrix might not be PSD due to finite shot noise.

if ur estimated kernel matrix has small negative eigenvalues, u have two options:

1. clip eigenvalues: compute eigendecomposition, set negative eigenvalues to zero, reconstruct
2. add regularization: replace K with $K + \lambda I$ for small $\lambda > 0$

mathematically, the eigenvalue clipping gives:

$$\tilde{K} = \sum_{i:\lambda_i > 0} \lambda_i \mathbf{v}_i \mathbf{v}_i^\top$$

where $\lambda_i, \mathbf{v}_i$ r the eigenvalues and eigenvectors of $\hat{K}$.

5.4 Shot Budget Analysis

lets work out the total computational cost:

$$\text{total circuits} = \frac{M(M-1)}{2} \tag{1}$$

$$\text{total shots} = \frac{M(M-1)}{2} \cdot N_s \tag{2}$$

$$\text{total gate operations} = \frac{M(M-1)}{2} \cdot N_s \cdot 2D \tag{3}$$

where D is the depth of the feature map circuit (the kernel circuit has depth $2D$).

Example 5.2. for $M = 50$ training points, $N_s = 1024$ shots, feature map depth $D = 20$:

- circuits: $50 \times 49/2 = 1225$
- total shots: $1225 \times 1024 = 1{,}254{,}400$
- if each shot takes $1\mu s$: total time ≈ 1.25 seconds

on a simulator, this is fast. on real hardware with queue times, this could take hours.

6 Quantum Kernels with Scikit-Learn

the beauty of the kernel approach is that once u have the kernel matrix, u can use standard classical ML tools. qiskit machine learning provides wrappers that make this seamless.

6.1 QSVC: Quantum Support Vector Classifier

QSVC wraps sklearn's `SVC` with a precomputed quantum kernel. the pipeline in qiskit:

1. create a feature map: e.g., `ZZFeatureMap(feature_dimension=2, reps=2)`
2. create the kernel: `FidelityQuantumKernel(feature_map=fm)`
3. create QSVC: `QSVC(quantum_kernel=kernel)`
4. fit: `qsvc.fit(X_train, y_train)`
5. predict: `qsvc.predict(X_test)`

internally, `fit()` computes the $M_{\text{train}} \times M_{\text{train}}$ kernel matrix and passes it to sklearn's SVC with `kernel='precomputed'`. `predict()` computes the $M_{\text{test}} \times M_{\text{train}}$ kernel matrix between test and training points.

Definition 6.1 (SVM with Precomputed Kernel). given a kernel matrix K where $K_{ij} = \kappa(\mathbf{x}_i, \mathbf{x}_j)$, the SVM optimization problem is:

$$\max_{\alpha} \sum_{i=1}^{M} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{M} \alpha_i \alpha_j y_i y_j K_{ij}$$

subject to $0 \leq \alpha_i \leq C$ and $\sum_i \alpha_i y_i = 0$.

the decision function for a new point $\mathbf{x}$ is:

$$f(\mathbf{x}) = \text{sign}\left(\sum_{i \in SV} \alpha_i y_i \kappa(\mathbf{x}, \mathbf{x}_i) + b \right)$$

where SV is the set of support vectors.

6.2 QSVR: Quantum Support Vector Regressor

same idea but for regression. uses sklearn's `SVR` under the hood.

the pipeline is identical except u use `QSVR(quantum_kernel=kernel)` and ur labels r continuous values instead of classes.

the SVR optimization uses the ε-insensitive loss:

$$\min_{\mathbf{w},b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^{M} \max(0, |y_i - f(\mathbf{x}_i)| - \varepsilon)$$

in the kernel formulation (dual):

$$\max_{\alpha,\alpha^*} -\frac{1}{2} \sum_{i,j} (\alpha_i - \alpha_i^*)(\alpha_j - \alpha_j^*) K_{ij} - \varepsilon \sum_i (\alpha_i + \alpha_i^*) + \sum_i y_i (\alpha_i - \alpha_i^*)$$

subject to $0 \leq \alpha_i, \alpha_i^* \leq C$ and $\sum_i (\alpha_i - \alpha_i^*) = 0$.

6.3 The Qiskit Code Flow

lets walk through wt happens step by step when u run QSVC (without showing actual code — just the logic flow):

1. initialization: u create a `ZZFeatureMap` specifying 2 qubits and 2 repetitions. this creates a parameterized circuit with parameters bound to data features.
2. kernel creation: the `FidelityQuantumKernel` wraps this feature map. it also creates a `ComputeUncompute` fidelity estimator internally.
3. training data: u prepare ur training data $X \in \mathbb{R}^{M \times d}$ and labels y.
4. kernel matrix computation: when u call `fit()`, the kernel object:
   - iterates over all unique pairs (i, j) with $i \leq j$
   - for each pair, binds $\mathbf{x}_i$ and $\mathbf{x}_j$ into the circuit parameters
   - constructs the compute-uncompute circuit: $U(\mathbf{x}_j)$ followed by $U^\dagger(\mathbf{x}_i)$
   - batches circuits together for efficient execution
   - runs all circuits on the backend (simulator or hardware) with specified shots
   - extracts the all-zeros probability from each result
   - fills in the kernel matrix symmetrically
5. SVM training: the kernel matrix is passed to sklearn's `SVC(kernel='precomputed').fit(K_train, y_train)`.
6. prediction: for new test points, compute the rectangular kernel matrix K_{test} where $(K_{\text{test}})_{ij} = \kappa(\mathbf{x}_i^{\text{test}}, \mathbf{x}_j^{\text{train}})$, then call sklearn's predict.

7 Kernel Alignment

how do u know if ur quantum kernel is good for a particular task? this is where kernel alignment comes in.

7.1 Definition

Definition 7.1 (Kernel-Target Alignment). given a kernel matrix K and target labels $\mathbf{y}$, the kernel-target alignment is:

$$A(K, \mathbf{y}\mathbf{y}^\top) = \frac{\langle K, \mathbf{y}\mathbf{y}^\top \rangle_F}{\|K\|_F \|\mathbf{y}\mathbf{y}^\top\|_F}$$

where $\langle A, B \rangle_F = \sum_{ij} A_{ij} B_{ij}$ is the frobenius inner product and $\|A\|_F = \sqrt{\langle A, A \rangle_F}$.

Intuition. the ideal kernel matrix for binary classification has $K_{ij} = 1$ when $y_i = y_j$ (same class) and $K_{ij} = 0$ when $y_i \neq y_j$ (different class). kernel alignment measures how close ur actual kernel matrix is to this ideal. values range from -1 to 1, with higher being better.

for binary labels $y_i \in \{-1, +1\}$, the ideal kernel matrix is:

$$K_{ij}^* = y_i y_j = \begin{cases} +1 & \text{if } y_i = y_j \\ -1 & \text{if } y_i \neq y_j \end{cases}$$

expanded:

$$A(K, \mathbf{y}\mathbf{y}^\top) = \frac{\sum_{ij} K_{ij} y_i y_j}{\sqrt{\sum_{ij} K_{ij}^2} \cdot \sqrt{\sum_{ij} (y_i y_j)^2}}$$

since $y_i y_j \in \{-1, +1\}$, we have $\sum_{ij} (y_i y_j)^2 = M^2$, so:

$$A = \frac{\sum_{ij} K_{ij} y_i y_j}{M \sqrt{\sum_{ij} K_{ij}^2}}$$

7.2 Why Alignment Matters

Theorem 7.1 (Alignment and Generalization). cristianini et al. (2001) showed that kernels with high alignment to the target tend to produce classifiers with low generalization error. specifically, if $A(K, \mathbf{y}\mathbf{y}^\top) \geq \gamma > 0$, then the expected test error of the SVM trained with kernel K is bounded:

$$\mathbb{E}[\text{error}] \leq O\left(\frac{1}{\gamma^2 M}\right)$$

Key Point. kernel alignment gives u a quick way to evaluate a quantum kernel without training an SVM. compute the kernel matrix, compute alignment, and if its low, try a different feature map. this saves a ton of time compared to doing full SVM training and cross-validation for each feature map choice.

7.3 Optimizing Kernel Alignment

u can optimize the feature map parameters to maximize kernel alignment. if ur feature map has trainable parameters $\boldsymbol{\theta}$ (in addition to data encoding), u can do:

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}} A(K(\boldsymbol{\theta}), \mathbf{y}\mathbf{y}^\top)$$

this is sometimes called "kernel alignment training" or "quantum kernel training". the gradient of alignment with respect to $\boldsymbol{\theta}$ can be estimated using the parameter shift rule.

the gradient of the alignment:

$$\frac{\partial A}{\partial \boldsymbol{\theta}} = \frac{1}{\|K\|_F \|\mathbf{y}\mathbf{y}^\top\|_F} \left[\frac{\partial \langle K, \mathbf{y}\mathbf{y}^\top \rangle_F}{\partial \boldsymbol{\theta}} - A \cdot \frac{\langle K, \frac{\partial K}{\partial \boldsymbol{\theta}} \rangle_F}{\|K\|_F^2} \cdot \|K\|_F \|\mathbf{y}\mathbf{y}^\top\|_F \right] \tag{4}$$

simplifying (using the chain rule and the fact that $\frac{\partial \|K\|_F}{\partial \boldsymbol{\theta}} = \frac{\langle K, \frac{\partial K}{\partial \boldsymbol{\theta}} \rangle_F}{\|K\|_F}$):

$$\frac{\partial A}{\partial \boldsymbol{\theta}} = \frac{1}{\|K\|_F \|\mathbf{y}\mathbf{y}^\top\|_F} \left[\sum_{ij} y_i y_j \frac{\partial K_{ij}}{\partial \boldsymbol{\theta}} - A \cdot \frac{\sum_{ij} K_{ij} \frac{\partial K_{ij}}{\partial \boldsymbol{\theta}}}{\|K\|_F^2} \cdot \|K\|_F \|\mathbf{y}\mathbf{y}^\top\|_F \right]$$

in practice, u use a quantum-classical optimization loop: compute kernel matrix, compute alignment and gradient, update parameters, repeat.

8 Practical Considerations

8.1 Number of Qubits

the number of qubits n should match the dimensionality of ur (preprocessed) data. if ur data has d features and $d > n$, u need dimensionality reduction (PCA is the standard choice). if $d < n$, u can:

- use only d qubits
- repeat the encoding: map each feature to multiple qubits
- add polynomial features and use the extra qubits for those

Key Point. more qubits doesnt always mean a better kernel. adding qubits increases the hilbert space dimension exponentially (2^n), but it also makes the kernel values concentrate around zero for random data pairs. this is a form of the "curse of dimensionality" in quantum feature spaces.

the concentration phenomenon: for n qubits and random data, the expected kernel value between two random points is:

$$\mathbb{E}_{\mathbf{x},\mathbf{x}'}[\kappa(\mathbf{x},\mathbf{x}')] \approx \frac{1}{2^n}$$

this means that for $n = 10$ qubits, typical kernel values r around $1/1024 \approx 0.001$. u need many shots just to distinguish this from zero.

8.2 Circuit Depth

the depth of the feature map affects both expressivity and noise sensitivity:

- shallow circuits (1-2 reps): less expressive but more noise-robust. the kernel circuit has depth $2 \times$ feature map depth.
- deep circuits (3+ reps): more expressive but very sensitive to noise. on current hardware, circuits deeper than about 50-100 layers r basically useless.

the noise impact on kernel quality: if each gate has error rate ε, and the kernel circuit has G gates total, the expected fidelity loss is approximately:

$$\kappa_{\text{noisy}}(\mathbf{x},\mathbf{x}') \approx (1-\varepsilon)^G \cdot \kappa_{\text{ideal}}(\mathbf{x},\mathbf{x}') + \frac{1-(1-\varepsilon)^G}{2^n}$$

this shows that noise pushes all kernel values toward $1/2^n$ (the maximally mixed state). the kernel loses its discriminative power.

8.3 Noise Impact on Kernel Quality

lets be more precise about how noise affects the kernel matrix:

Definition 8.1 (Noisy Kernel). under a global depolarizing noise model with parameter p (probability of depolarization per gate), the noisy kernel is:

$$\kappa_{\text{noisy}} = (1-p)^{2D_{\text{gates}}} \cdot \kappa_{\text{ideal}} + (1-(1-p)^{2D_{\text{gates}}}) \cdot \frac{1}{2^n}$$

where D_{gates} is the total number of gates in the feature map circuit (the kernel circuit has $2D_{\text{gates}}$ gates bcs of compute-uncompute).

Example 8.1. for a 4-qubit ZZFeatureMap with 2 reps, roughly 30 gates per rep, so $D_{\text{gates}} \approx 60$. with gate error $p = 0.001$:

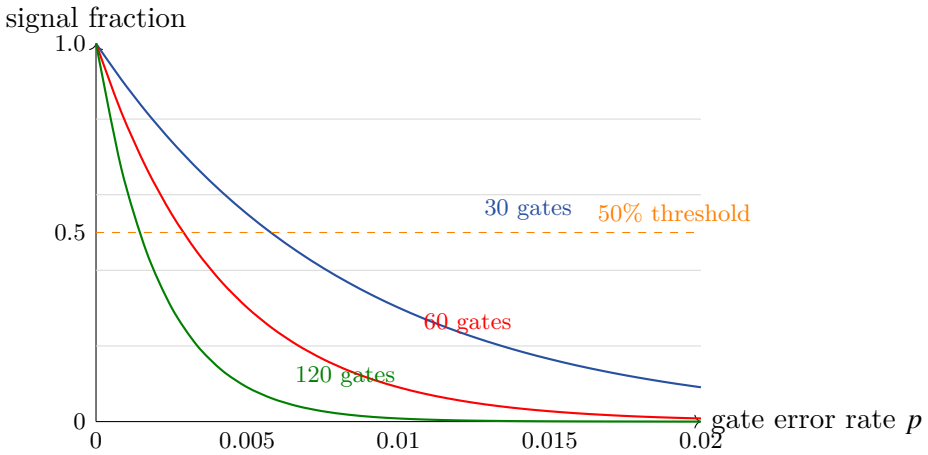
$$(1 - 0.001)^{120} \approx e^{-0.12} \approx 0.887$$

so the noisy kernel is about 89% of the ideal kernel plus 11% noise floor. thats actually not terrible.

but with gate error $p = 0.01$:

$$(1 - 0.01)^{120} \approx e^{-1.2} \approx 0.301$$

now the noisy kernel is only 30% signal and 70% noise. the kernel matrix becomes almost useless.



9 Benchmarking: Quantum vs Classical Kernels

lets see how quantum kernels actually perform on standard toy datasets. spoiler: on these small datasets, classical kernels r competitive or better. but the exercise is still valuable bcs it shows the full pipeline.

9.1 The Iris Dataset

the iris dataset: 150 samples, 4 features, 3 classes. for quantum kernels, we typically use 2 classes (setosa vs versicolor, or versicolor vs virginica) and 2 features (for 2 qubits).

setup: 2 qubits, ZZFeatureMap with 2 reps, 1024 shots. train/test split 70/30. compare against RBF kernel SVM.

expected results:

Kernel	Train Accuracy	Test Accuracy
Quantum (ZZFeatureMap, reps=2)	$\sim 95\%$	$\sim 93\%$
Classical (RBF, $\gamma = 1.0$)	$\sim 97\%$	$\sim 95\%$
Classical (polynomial, deg=3)	$\sim 96\%$	$\sim 93\%$
Classical (linear)	$\sim 88\%$	$\sim 87\%$

the quantum kernel is competitive with the polynomial kernel but slightly below RBF. this is typical — on easy datasets with few features, classical kernels have been optimized for decades and work great.

9.2 The Moons Dataset

make_moons from sklearn: 200 samples, 2 features, 2 classes, with noise=0.2. this is a classic nonlinear classification problem.

setup: 2 qubits, ZZFeatureMap with 2 reps, 1024 shots.

expected results:

Kernel	Train Accuracy	Test Accuracy
Quantum (ZZFeatureMap, reps=1)	$\sim 85\%$	$\sim 83\%$
Quantum (ZZFeatureMap, reps=2)	$\sim 92\%$	$\sim 90\%$
Quantum (ZZFeatureMap, reps=3)	$\sim 95\%$	$\sim 91\%$
Classical (RBF, tuned)	$\sim 97\%$	$\sim 95\%$

Intuition. on moons, the quantum kernel with enough reps gets close to RBF but doesnt beat it. the reason: the moons dataset has a simple nonlinear structure that RBF captures perfectly. the quantum kernel introduces more complex features that r partly relevant but also partly noise.

9.3 The Circles Dataset

make_circles from sklearn: 200 samples, 2 features, 2 classes, with noise=0.15. concentric circles — another classic nonlinear problem.

expected results:

Kernel	Train Accuracy	Test Accuracy
Quantum (ZZFeatureMap, reps=2)	$\sim 88\%$	$\sim 86\%$
Quantum (ZFeatureMap, reps=2)	$\sim 75\%$	$\sim 73\%$
Classical (RBF)	$\sim 95\%$	$\sim 93\%$
Classical (polynomial, deg=2)	$\sim 93\%$	$\sim 91\%$

Key Point. the circles dataset shows why ZFeatureMap (no entanglement) is bad — it cant capture the radial structure. ZZFeatureMap does better bcs the entangling gates create features that can separate circles, but RBF is still better bcs its literally designed for radial problems ($k(\mathbf{x}, \mathbf{x}') = e^{-\gamma\|\mathbf{x}-\mathbf{x}'\|^2}$).

9.4 Interpreting Kernel Matrices

wt does a good vs bad kernel matrix look like?

good kernel matrix:

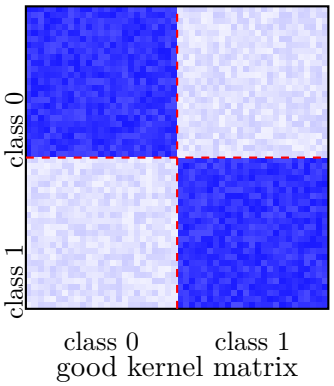
- clear block structure: points in the same class have high kernel values, points in different classes have low kernel values

- smooth variation: nearby points (in the same class) have similar kernel values
- well-conditioned: eigenvalues span a reasonable range, not all clustered near zero

bad kernel matrix:

- no block structure: kernel values r random regardless of class labels
- diagonal dominance: $K_{ii} \gg K_{ij}$ for all $i \neq j$ (the kernel is too "sharp")
- all entries similar: $K_{ij} \approx c$ for all i, j (the kernel doesnt discriminate, often happens with too many qubits)
- heavy noise: lots of random variation in entries

conceptually, a kernel matrix heatmap for binary classification should look smtg like this:



good kernel matrix

10 Results Interpretation Guidelines

when u run quantum kernel experiments, here is how to interpret wt u see:

10.1 Kernel Matrix Diagnostics

1. check the diagonal: should all be 1.0 (or very close). if not, smtg is wrong with ur implementation.
2. check the off-diagonal range: values should span a meaningful range, say $[0.01, 0.8]$. if everything is near 0, ur kernel is too "spread out" (too many qubits or too much entanglement). if everything is near 1, ur kernel is too "narrow" (features r not being differentiated).
3. compute alignment: alignment above 0.1 is decent for a random feature map. above 0.3 is good. above 0.5 is excellent.
4. eigenvalue spectrum: compute eigenvalues of K. a good kernel has a few large eigenvalues and many small ones (low effective rank). this means the kernel is capturing a few important directions in feature space. if all eigenvalues r similar, the kernel is essentially random.
5. compare with ideal: compute the ideal kernel $K^* = \mathbf{y}\mathbf{y}^\top$ and visually compare. the closer ur kernel looks to this, the better.

10.2 When Quantum Kernels Work Well

based on current research, quantum kernels tend to work well when:

- the data has structure that maps naturally to quantum correlations (e.g., data generated by quantum processes)
- the feature dimension matches the number of available qubits
- the dataset is small enough that $O(M^2)$ kernel evaluations r feasible
- the circuit depth is manageable given hardware noise

10.3 When Quantum Kernels Struggle

- large datasets ($M > 500$): the $O(M^2)$ cost becomes prohibitive
- high-dimensional data requiring many qubits: kernel concentration kills performance
- noisy hardware with deep circuits: noise washes out the signal
- problems where classical kernels (especially RBF) already work perfectly

11 Advanced: Projected Quantum Kernels

one way to mitigate the kernel concentration problem is to use projected quantum kernels (huang et al., 2021).

Definition 11.1 (Projected Quantum Kernel). instead of computing the full state fidelity, project the quantum state onto local subsystems:

$$\kappa_{\text{proj}}(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma \sum_S \left\| \rho_S(\mathbf{x}) - \rho_S(\mathbf{x}') \right\|_F^2\right)$$

where $\rho_S(\mathbf{x}) = \text{Tr}_{\bar{S}}[|\phi(\mathbf{x})\rangle\langle\phi(\mathbf{x})|]$ is the reduced density matrix on subsystem S.

the key differences from standard quantum kernels:

1. measures local properties (1 or 2 qubit reduced states) instead of global fidelity
2. doesnt suffer from exponential concentration bcs local observables have $O(1)$ variance
3. can be computed from local measurements: measure each qubit (or pair) in X, Y, Z bases
4. the resulting kernel is guaranteed to be PSD (its a gaussian kernel on measured features)

the local measurement approach:

$$\rho_i(\mathbf{x}) \to (\langle X_i \rangle, \langle Y_i \rangle, \langle Z_i \rangle) \in \mathbb{R}^3 \tag{5}$$

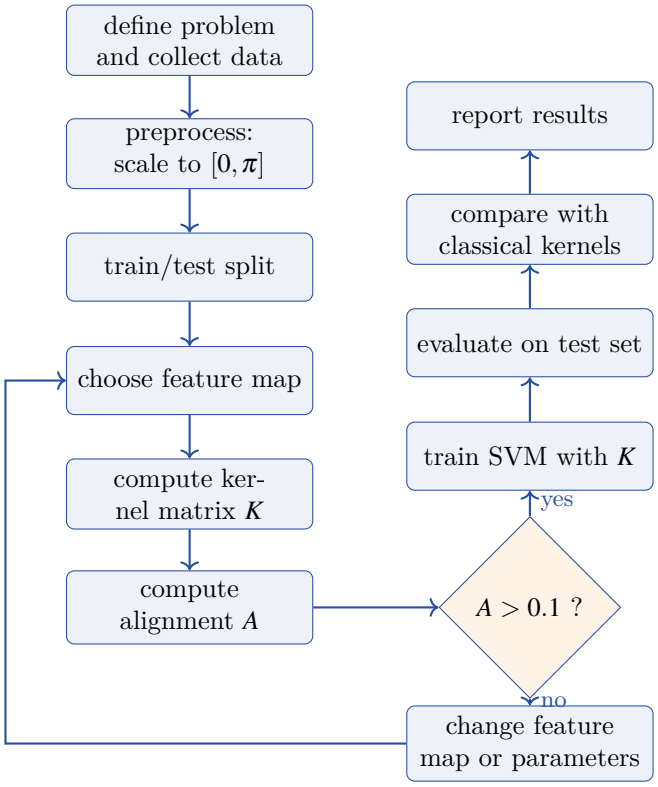
$$\rho_{ij}(\mathbf{x}) \to (\langle X_i X_j \rangle, \langle X_i Y_j \rangle, \ldots, \langle Z_i Z_j \rangle) \in \mathbb{R}^9 \tag{6}$$

this gives a classical feature vector of dimension $3n + 9\binom{n}{2}$ that captures the essential quantum information without the concentration problem.

Intuition. projected quantum kernels r like a compromise: u use the quantum circuit to create complex features, but then u extract classical summary statistics from the quantum state. its less "quantum" than full fidelity kernels, but it actually works better in practice on noisy hardware.

12 Complete Workflow Diagram

here is the complete workflow for a quantum kernel experiment, showing all the steps from problem definition to results:



13 Benchmarking Strategy

when benchmarking quantum kernels, u need a systematic approach. here is the recommended protocol:

13.1 Experimental Setup

1. datasets: use at least 3 datasets with different characteristics (linear, nonlinear, multi-class reduced to binary)
2. quantum kernels to test:
   - ZFeatureMap (baseline, should match classical)
   - ZZFeatureMap with reps = 1, 2, 3
   - PauliFeatureMap with different paulis
3. classical kernels to compare:
   - linear kernel (baseline)
   - RBF kernel with tuned γ
   - polynomial kernel with tuned degree

4. metrics: accuracy, F1-score, kernel alignment, training time, total quantum cost (circuits × shots)

5. repetitions: run each experiment 5-10 times with different random train/test splits to get error bars

13.2 Fair Comparison Guidelines

Key Point. a fair comparison requires:

- same train/test splits for quantum and classical
- hyperparameter tuning for both quantum (reps, entanglement) and classical (C, γ, degree)
- same evaluation metrics
- reporting of computational cost (quantum kernels r much slower)

dont cherry-pick the one run where quantum beats classical. report averages and standard deviations.

14 Common Pitfalls

from experience (and from reading a lot of quantum kernel papers), here r the most common mistakes:

1. forgetting to normalize: raw features in different ranges lead to rotation gates that dont use the full bloch sphere. always preprocess.

2. too many qubits: using 10+ qubits for a dataset with 2 useful features. the extra qubits just add noise and cause concentration.

3. too many reps: more reps = more expressivity, but also more noise sensitivity and slower execution. start with reps=2.

4. not enough shots: below 512 shots, the kernel matrix is dominated by statistical noise. use at least 1024.

5. not checking PSD: the estimated kernel matrix might not be positive semi-definite. sklearn's SVM will still run but might give weird results. always check and fix.

6. comparing unfairly: comparing an untuned quantum kernel against an untuned classical kernel (or worse, against a tuned one). always tune both.

7. ignoring the cost: claiming quantum is "comparable" to classical while ignoring that the quantum kernel took 1000x more compute. always report cost.

8. overfitting with deep kernels: very expressive quantum kernels can perfectly fit the training data but generalize poorly. monitor the train/test gap.

15 Summary and Key Takeaways

this lecture covered the practical side of quantum kernels. lets recap:

1. the pipeline is straightforward: data → normalize → feature map → kernel matrix → SVM → predictions. qiskit's `FidelityQuantumKernel` handles the quantum parts.
2. feature map choice matters a lot: ZFeatureMap gives u a classical kernel. ZZFeatureMap is the standard for potentially quantum advantage. entanglement is necessary but not sufficient.
3. kernel computation is expensive: $O(M^2)$ circuits, each with many shots. the total cost scales quadratically with dataset size.
4. noise degrades kernel quality: the signal fraction drops exponentially with circuit depth and gate error rate. shallow circuits r preferable on current hardware.
5. kernel alignment is ur best friend: it tells u quickly whether a kernel is good for ur task without doing full SVM training.
6. on toy datasets, classical wins or ties: this is expected. the real question is whether quantum kernels can solve problems that classical kernels genuinely cant (next lecture).
7. projected quantum kernels: a practical compromise that avoids concentration and works better on noisy hardware.

next lecture well go deeper into QSVC, QSVR, and the pegasos algorithm, plus tackle real datasets and do proper cross-validation and hyperparameter tuning.